

lux Connection-Lease Reaping — Fidelity Control (Cascade Incomplete)

Buggy variant: `TimedReapEnd` and `Claim` release the registry and ownership but never clear subscri

September 2026

1 Introduction

This is the fidelity control for round 3's I7 in `docs/connection_lease_reaping.tex`, not the design. It removes only the three new cascade-clearing terms `TimedReapEnd` and `Claim` otherwise carry: the registry-and-ownership legs are released exactly as the fixed spec releases them, but `hasSubs`, `hasWriter`, and `inboxNonEmpty` are left untouched. `_depart_lapsed` and `apply`'s embedded sweep never call `Hub.on_disconnect` or the transport-owned sink today – a timer-reaped or swept-aside connection keeps its subscriptions, writer binding, and inbox queue forever, even though the identical connection is already gone from the registry and owns nothing. Read the main specification first; this document states only what differs, in Section 5.1 and Section 5.2.

2 Basic Types

$[CONN, IDENT, SCENE]$

$TSTATE ::= tup \mid tdown$

$LSTATE ::= llive \mid llapsed$

$OWNER ::= unowned \mid conn\langle\langle CONN \rangle\rangle$

$FSTATE ::= fclear \mid fset$

Unchanged from the main specification.

3 State

ConnReg

$registered : \mathbb{P} CONN$
 $identity : CONN \leftrightarrow IDENT$
 $transport : CONN \leftrightarrow TSTATE$
 $lease : CONN \leftrightarrow LSTATE$
 $owner : SCENE \rightarrow OWNER$
 $hasSubs : CONN \rightarrow FSTATE$
 $hasWriter : CONN \rightarrow FSTATE$
 $inboxNonEmpty : CONN \rightarrow FSTATE$
 $cascadeFor : \mathbb{P} CONN$

$registered \subseteq \text{dom } identity$
 $\text{dom } identity = \text{dom } transport$
 $\text{dom } transport = \text{dom } lease$
 $\#cascadeFor \leq 1$

Unchanged from the main specification.

4 Initialization

<i>Init</i>
<i>ConnReg'</i>
$registered' = \emptyset$ $identity' = \emptyset$ $transport' = \emptyset$ $lease' = \emptyset$ $owner' = (\lambda s : SCENE \bullet unowned)$ $hasSubs' = (\lambda c : CONN \bullet fclear)$ $hasWriter' = (\lambda c : CONN \bullet fclear)$ $inboxNonEmpty' = (\lambda c : CONN \bullet fclear)$ $cascadeFor' = \emptyset$

Unchanged from the main specification.

5 Operations

<i>Connect</i>
$\Delta ConnReg$ $c? : CONN$ $i? : IDENT$
$cascadeFor = \emptyset$ $registered' = registered \cup \{c?\}$ $identity' = identity \oplus \{c? \mapsto i?\}$ $transport' = transport \oplus \{c? \mapsto tup\}$ $lease' = lease \oplus \{c? \mapsto llive\}$ $hasWriter' = hasWriter \oplus \{c? \mapsto fset\}$ $hasSubs' = hasSubs$ $inboxNonEmpty' = inboxNonEmpty$ $cascadeFor' = cascadeFor$ $owner' = owner$

<i>Renew</i>
$\Delta ConnReg$ $c? : CONN$
$c? \in registered$ $transport\ c? = tup$ $cascadeFor = \emptyset$ $lease' = lease \oplus \{c? \mapsto llive\}$ $registered' = registered$ $identity' = identity$ $transport' = transport$ $hasSubs' = hasSubs$ $hasWriter' = hasWriter$ $inboxNonEmpty' = inboxNonEmpty$ $cascadeFor' = cascadeFor$ $owner' = owner$

Activate

$\Delta ConnReg$
 $c? : CONN$

$c? \in registered$
 $transport\ c? = tup$
 $hasSubs' = hasSubs \oplus \{c? \mapsto fset\}$
 $hasWriter' = hasWriter \oplus \{c? \mapsto fset\}$
 $inboxNonEmpty' = inboxNonEmpty \oplus \{c? \mapsto fset\}$
 $registered' = registered$
 $identity' = identity$
 $transport' = transport$
 $lease' = lease$
 $owner' = owner$
 $cascadeFor' = cascadeFor$

TransportDies

$\Delta ConnReg$
 $c? : CONN$

$c? \in registered$
 $transport\ c? = tup$
 $transport' = transport \oplus \{c? \mapsto tdown\}$
 $registered' = registered$
 $identity' = identity$
 $lease' = lease$
 $owner' = owner$
 $hasSubs' = hasSubs$
 $hasWriter' = hasWriter$
 $inboxNonEmpty' = inboxNonEmpty$
 $cascadeFor' = cascadeFor$

LeaseLapse

$\Delta ConnReg$
 $c? : CONN$

$c? \in registered$
 $lease\ c? = llive$
 $lease' = lease \oplus \{c? \mapsto llapsed\}$
 $registered' = registered$
 $identity' = identity$
 $transport' = transport$
 $owner' = owner$
 $hasSubs' = hasSubs$
 $hasWriter' = hasWriter$
 $inboxNonEmpty' = inboxNonEmpty$
 $cascadeFor' = cascadeFor$

Read

$\Xi ConnReg$

<i>Depart</i> $\Delta ConnReg$ $c? : CONN$
$c? \in registered$ $registered' = registered \setminus \{c?\}$ $owner' = owner \oplus$ $(\{s : SCENE \mid owner\ s = conn(c?)\} \times \{unowned\})$ $hasSubs' = hasSubs \oplus \{c? \mapsto fclear\}$ $hasWriter' = hasWriter \oplus \{c? \mapsto fclear\}$ $inboxNonEmpty' = inboxNonEmpty \oplus \{c? \mapsto fclear\}$ $identity' = identity$ $transport' = transport$ $lease' = lease$ $cascadeFor' = cascadeFor$

Depart is unchanged from the fixed spec, and correct: this control isolates the gap to **TimedReapEnd** and **Claim** alone, matching the design document's own report that **Depart**'s real-code counterpart already reaches all four cascade legs once fixed.

<i>TimedReapBegin</i> $\Delta ConnReg$ $c? : CONN$
$c? \in registered$ $lease\ c? = lapsed$ $cascadeFor = \emptyset$ $registered' = registered \setminus \{c?\}$ $owner' = owner \oplus$ $(\{s : SCENE \mid owner\ s = conn(c?)\} \times \{unowned\})$ $cascadeFor' = \{c?\}$ $identity' = identity$ $transport' = transport$ $lease' = lease$ $hasSubs' = hasSubs$ $hasWriter' = hasWriter$ $inboxNonEmpty' = inboxNonEmpty$

Unchanged from the fixed spec.

5.1 TimedReapEnd — BUGGY: releases the lock, never the cascade

<i>TimedReapEnd</i> $\Delta ConnReg$ $c? : CONN$
$cascadeFor = \{c?\}$ $cascadeFor' = \emptyset$ $hasSubs' = hasSubs$ $hasWriter' = hasWriter$ $inboxNonEmpty' = inboxNonEmpty$ $registered' = registered$ $identity' = identity$ $transport' = transport$ $lease' = lease$ $owner' = owner$

The three terms that clear **hasSubs**/**hasWriter**/**inboxNonEmpty** in the fixed spec's **TimedReapEnd** are replaced here with the identity – the cascade window closes (**cascadeFor** still clears), but nothing the window was meant to release actually is.

5.2 Claim — BUGGY: sweeps the registry and ownership, never the cascade

Claim

$\Delta ConnReg$

$c? : CONN$

$s? : SCENE$

$c? \in registered$

$transport\ c? = tup$

$lease' = lease \oplus \{c? \mapsto llive\}$

$registered' = registered \setminus$

$\{c2 : CONN \mid c2 \in registered \wedge c2 \neq c? \wedge lease\ c2 = llapsed\}$

$(owner \oplus (\{s : SCENE \mid \exists c2 : CONN \bullet$
 $c2 \in registered \wedge c2 \neq c? \wedge lease\ c2 = llapsed \wedge owner\ s = conn(c2)\}$
 $\times \{unowned\}))\ s? = unowned$

$\vee$

$(owner \oplus (\{s : SCENE \mid \exists c2 : CONN \bullet$
 $c2 \in registered \wedge c2 \neq c? \wedge lease\ c2 = llapsed \wedge owner\ s = conn(c2)\}$
 $\times \{unowned\}))\ s? = conn(c?)$

$owner' = (owner \oplus (\{s : SCENE \mid \exists c2 : CONN \bullet$
 $c2 \in registered \wedge c2 \neq c? \wedge lease\ c2 = llapsed \wedge owner\ s = conn(c2)\}$
 $\times \{unowned\})) \oplus \{s? \mapsto conn(c?)\}$

$hasSubs' = hasSubs$

$hasWriter' = hasWriter$

$inboxNonEmpty' = inboxNonEmpty$

$identity' = identity$

$transport' = transport$

$cascadeFor' = cascadeFor$

The registry-and-ownership sweep is exactly the fixed spec's: same set, same $c2 \neq c?$ exclusion. Only the three cascade-clearing terms are replaced with the identity.

6 Verification: the goal that must flip

Against this variant, I7's negation must be *found* – the trace in `connection_lease_reaping.tex`, Section “Cascade-incomplete,” reaches it in six steps: `Connect(c1, i1)`; `Activate(c1)`; `TransportDies(c1)`; `LeaseLapse(c1)`; `TimedReapBegin(c1)`; `TimedReapEnd(c1)`.

```
# must be FOUND against this variant (NOT found against the fixed spec)
probccli docs/connection_lease_reaping_cascade_incomplete_buggy.tex \
-model_check -nodead \
-goal "#c.(c:CONN & not(c : registered) & not(c : cascadeFor) &
      (hasSubs(c) = fset or hasWriter(c) = fset or
       inboxNonEmpty(c) = fset))" \
-p DEFAULT_SETSIZE 3
```

The identical goal is also reachable through Claim's half of the same gap, with no `TimedReapBegin`/`TimedReapEnd` in the trace at all: `Connect(c1, i1)`; `Activate(c1)`; `LeaseLapse(c1)`; `Connect(c2, i2)`; `Claim(c2, s2)` – the last step's sweep removes `c1` and releases its ownership, but leaves its three cascade flags untouched.

I2/I3/I5/I6 must stay unaffected – this variant does not touch `registered` or `owner` differently from the fixed spec anywhere:

```
# must remain NOT found
probccli docs/connection_lease_reaping_cascade_incomplete_buggy.tex \
-model_check -nodead \
-goal "#s.(#c.(s:SCENE & c:CONN & owner(s) = conn(c) &
      not(c : registered)))" \
-p DEFAULT_SETSIZE 3
```

I1 must stay unaffected – this variant does not touch `TransportDies`, `LeaseLapse`, or `TimedReapBegin`, so the chain that establishes I1's positive outcome still completes:

```
# must remain FOUND
probccli docs/connection_lease_reaping_cascade_incomplete_buggy.tex \
  -model_check -nodead \
  -goal "#c.(c:CONN & c : dom(identity) & not(c : registered) &
        transport(c) = tdown)" \
  -p DEFAULT_SETSIZE 3
```